CMAA5043 Creative Prototyping
Lab 4 Tutorial: Hook, Router

Welcome to *Creative Prototyping: Designing the Interactive Interface*! In this course, we'll dive into the core concepts of React, including Hook and Router. As we progress, we'll apply these concepts to refine our **personal website**, which will have more functions.

Task 1. Hook

1.1. What is Hook?

Hooks are functions that allow the use of state and other features within functional components. In older versions of React, components were divided into class components and functional components. Traditionally, functional components were simple but stateless, while class components were more complex and capable of managing their own state. However, with the use of Hooks, functional components can now manage their own state, making the code simpler and more intuitive. As a result, the components we write today are mostly functional components.

In fact, we have already applied many hooks in our previous work. For example, in lab 3, we used useState or useRef to manage the state in LatestNews and items displayed in Hubs. In this lab, we will systematically explore hooks, including both the **built-in hooks in React** and **custom hooks**.

Before we dive in, it's important to understand the general rules for using Hooks:

1. Hooks can only be called at the **top level** of a function component: "Top level" means the outermost scope of a React function component. Hooks should not be called inside conditionals, loops, nested functions, or within a return statement.

```jsx
const Hubs = ({ hub }) => {
    // 1. ❌ Wrong: Using Hook inside conditional statement.
    if (hub.length > 0) {
        const [startIndex, setStartIndex] = useState(0);
    }

    // 2. ❌ Wrong: Using Hook inside loop.
    for (let i = 0; i < hub.length; i++) {
        const [state, setState] = useState(i);
    }

    // 3. ❌ Wrong: Using Hook inside function.
    const goToRight = () => {
        const [count, setCount] = useState(0);

        if (startIndex < hub.length - 3) {
          setStartIndex(prev => prev + 1);
        }
    };
```

```jsx
return (
    <div>
        <h2>Our Hub</h2>
        <div style={styles.hubWrapper}>
            <div style={styles.HubGrid}>
                <button onClick={goToLeft} style={styles.arrowButton} disabled={startIndex === 0}>
                    ←
                </button>
                {currenthub.map((item, index) => (
                    // 4. ❌ Wrong: Using Hook inside return.
                    <div key={index} style={styles.hubItem} onClick={() => {
                        const [isClicked, setIsClicked] = useState(false);
                        setIsClicked(true);
                    }}>
                        <img src={item.imgSrc} alt={item.title} style={styles.hubImage} />
                        <p style={styles.hubTitle}>{item.title}</p>
                    </div>
                ))}
                <button onClick={goToRight} style={styles.arrowButton} disabled={startIndex === hub.length - 3}
                    →
                </button>
            </div>
        </div>
    </div>
);
```

Figure 1. The wrong case in hook use.

2. Hooks can only be called from **React function components or custom Hooks**. You can not call hooks from regular JavaScript functions or other types of functions outside of React's context.

```
function NormalJsFunction() {
    // wrong: using hook inside normal js function.
    const [count, setCount] = useState(0);
    return count;
}
```

Figure 2. The wrong case in hook use.

Following these rules ensures that Hooks behave predictably and maintain the integrity of your component state and behavior.

1.2. Built-in Hooks

1.2.1 State Hook

The State Hook allows the use of state in functional components. State refers to a snapshot of a component's data at a specific moment. During rendering, the component determines its appearance based on this state data. The changes to the state data will also cause the component to re-render.

In React, the syntax for the State Hook is as follows: the two items in the array on the left represent the **state data** and **the function** to update that state, respectively. On the right, `useState` represents the Hook, and `initialValue` represents the **initial value of the state**.

```
const [state, setState] = useState(initialValue);
```

The `initialValue` can be of various types, such as a `boolean, number, array, or even an object`. This flexibility allows you to initialize your state with the appropriate data type for your component's needs.

For the `setState` function, there are multiple ways to update the state:

1. **Direct Value Assignment**: We can directly assign a new value to the state.

```
const [state, setState] = useState(222);
setState(233); // Sets the state to 233
```

2. **Using the Previous State**: We can compute a new value based on the previous state.

```
setState(state + 1); // Adds 1 to the current state value
```

3. **Functional Updates:** For more complex updates, especially when the new state depends on the previous state, we can use a function to ensure the update is performed correctly and avoids potential race conditions.

```
setState(prevState => prevState + 1);
// Ensures the update uses the latest state value
```

When updating state based on the previous state, it's recommended to use the **functional updates (prevState => ...)** to ensure the update is consistent.

1.2.2. Effect Hook

The Effect Hook (useEffect) is a hook in React used to handle side effects. Side effects refer to operations that are **not directly related to rendering the UI**, such as data fetching (using APIs), subscriptions, logging, setting up timers, etc. The effect hook's syntax is:

```
useEffect(() => {
  // Side effect code

  // Optional cleanup function
  return () => {
    // Cleanup code
  };
}, [dependencies array]);
```

The first part is the side effect code, which contains the code to be executed. The second part is the cleanup function (optional), used to clean up resources (e.g., subscriptions or timers), and it is called before the component unmounts or the effect re-executed. The third part is the dependencies array (optional).

If no array is provided, the effect will be executed after every render. If an empty dependency array is provided, the effect will only execute once when the component mounts. If there are dependencies specified, the effect will execute whenever the dependencies change:

```
// 1. No dependencies array: Executes after every render
useEffect(() => {
  console.log('Executes on every render');
});

// 2. Empty dependencies array: Executes only once when the component mounts
useEffect(() => {
  console.log('Executes only once');
```

```
}, []);

// 3. Dependencies array with values: Executes when the dependencies change
useEffect(() => {
   console.log('Executes when dependencies change');
}, [someValue]);
```

We attempt to optimize the Hubs to implement a looping carousel and achieve automatic cycling through the useEffect. The logic for the looping carousel can be implemented simply using index % hub.length. For the automatic cycling, we can use useEffect to move the carousel one step to the right every 10 seconds. Here, the timer is set and cleaned up using setInterval and clearInterval. The dependency array is [hub.length], meaning that the useEffect will only re-execute if the number of items in the hub changes.

```
const Hubs = ({ hub }) => {
  const [startIndex, setStartIndex] = useState(0);
  useEffect(() => {
    const timer = setInterval(() => {
      setStartIndex(prev => (prev + 1) % hub.length);
    }, 10000);

    return () => clearInterval(timer);
  }, [hub.length]);

  const goToRight = () => {
    setStartIndex(prev => (prev + 1) % hub.length);
  };

  const goToLeft = () => {
    setStartIndex(prev => (prev - 1 + hub.length) % hub.length);
  };

  const getCurrentItems = () => {
    const items = [];
    for (let i = 0; i < 3; i++) {
      const index = (startIndex + i) % hub.length;
      items.push(hub[index]);
    }
    return items;
  };

  const currenthub = getCurrentItems();
```

Figure 3. The hubs component.

After applying the useEffect, we have implemented the automatic loop. However, on the website, we notice that the automatic looping and user interactions operate independently. If a user clicks just as the automatic loop is about to trigger, the loop will quickly continue, which could lead to an annoying user experience. Therefore, in the ref hook, we will explore how to solve the problem.

1.2.3 Ref Hook

The Ref Hook (useRef) is a hook in React used to store mutable values. **The mutable values are that can change without causing the component to re-render** (while the change of values in useState will trigger a re-render). It has two primary uses: **1. Accessing and manipulating DOM elements**. **2. Storing mutable values**. The syntax for the ref hook is shown below, and its value is accessed via .current.

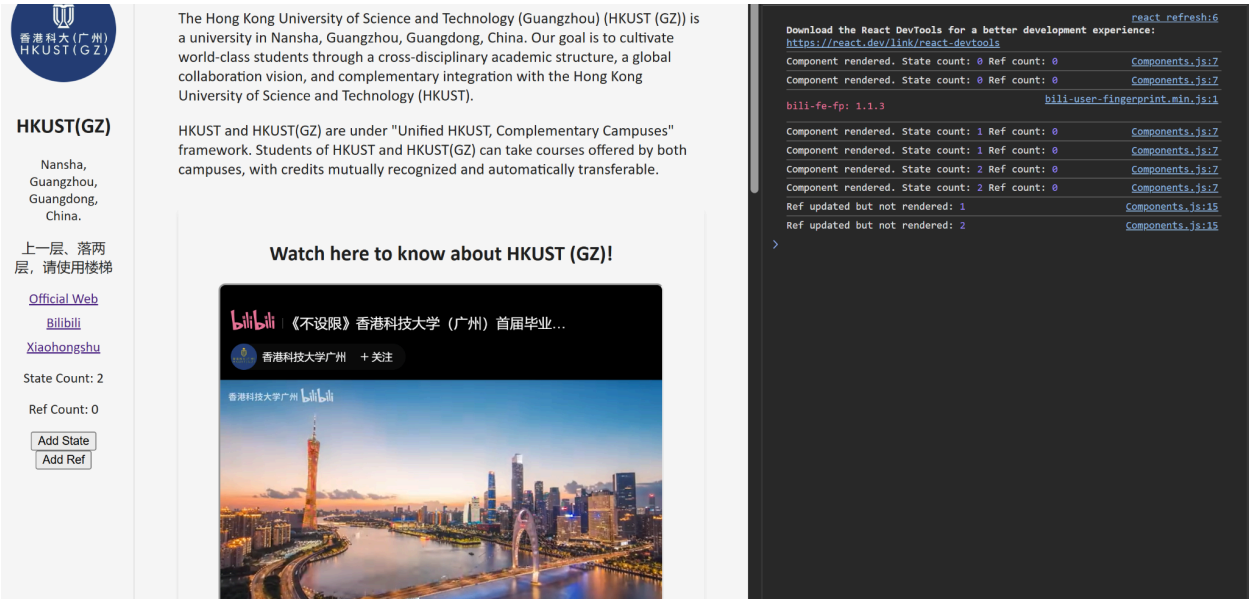
```
const refContainer = useRef(initialValue);
refContainer.current += 1
```

In the LeftProfile, we add two counters to demonstrate the difference between useRef and useState. When clicking "Add State" and "Add Ref", they update the state count and ref count respectively.

```jsx
const LeftProfile = () => {
  const [stateCount, setStateCount] = useState(0);
  const refCount = useRef(0);

  console.log('Component rendered. State count:', stateCount, 'Ref count:', refCount.current);

  const handleStateClick = () => {
    setStateCount(prev => prev + 1);
  };

  const handleRefClick = () => {
    refCount.current += 1;
    console.log('Ref updated but not rendered:', refCount.current);
  };
```

```jsx
return (
  <div style={styles.leftProfile}>
    <img src="hkust(gz).jpg" alt="School Badge" style={styles.profileImage} />
    <h2>HKUST(GZ)</h2>
    <p>Nansha, Guangzhou, Guangdong, China.</p>
    <p>上一层、落两层，请使用楼梯</p>
    <div style={styles.socialLinks}>
      <a href="https://www.hkust-gz.edu.cn/" target="_blank" rel="noreferrer">
        Official Web
      </a>
      <a href="https://space.bilibili.com/16783101" target="_blank" rel="noreferrer">
        Bilibili
      </a>
      <a href="https://www.xiaohongshu.com/user/profile/640574e60000000029014ff1" target="_blank" rel="noreferrer">
        Xiaohongshu
      </a>
    </div>
    <div>
      <p>State Count: {stateCount}</p>
      <p>Ref Count: {refCount.current}</p>
      <button onClick={handleStateClick}>Add State</button>
      <button onClick={handleRefClick}>Add Ref</button>
    </div>
  </div>
);
```

Figure 4, 5. The refined LeftProfile component.



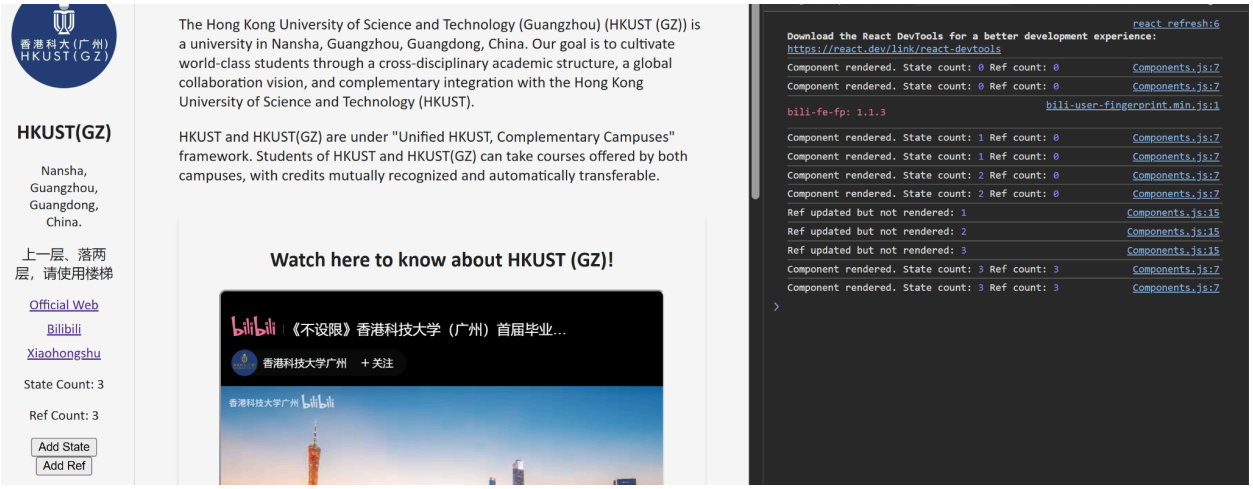


Figure 6, 7. The different results between useState and useRef.

If we open the browser's developer tools, we'll notice that the state count updates immediately, but when you click "Add Ref", the ref count does not update right away. When clicking "Add State" again, we'll see that the ref count suddenly reflects all the previous updates. This happens because ref does not trigger a re-render of the component; the ref value only updates when the component is re-rendered due to some other operation.

Going back to the Hubs, we want to reset the timer if the user clicks the left or right button. This prevents the page from changing quickly after the user interacts with it. To achieve this, we use useRef to store the timer and move the timer function outside. Whenever the user clicks goToRight or goToLeft, the existing timer is first cleared, and then the timer is reset using

setRotationTimer. During this process, since the timer is a mutable value, changing it does not cause the component to re-render.

```
const Hubs = ({ hub }) => {
  const [startIndex, setStartIndex] = useState(0);
  const timer = useRef(null);  // use ref hook to store the timer

  const setRotationTimer = () => {
    return setInterval(() => {
      setStartIndex(prev => (prev + 1) % hub.length);
    }, 10000);
  };

  useEffect(() => {
    timer.current = setRotationTimer();
    return () => clearInterval(timer.current);
  }, [hub.length]);

  const goToRight = () => {
    setStartIndex(prev => (prev + 1) % hub.length);
    clearInterval(timer.current);
    timer.current = setRotationTimer();
  };

  const goToLeft = () => {
    setStartIndex(prev => (prev - 1 + hub.length) % hub.length);
    clearInterval(timer.current);
    timer.current = setRotationTimer();
  };
```

Figure 8. The refined Hubs Component.

In HCI research and design, we always need to capture user behavior for analysis, which can also be achieved by useRef. For example, if we want to know whether the user's mouse has entered the videoPlayer to reflect their attention and interest on the website, we can make the following modifications:

```
const VideoPlayer = ({ isDarkMode }) => {
  const videoRef = useRef(null);
  const [isHovered, setIsHovered] = useState(false);

  useEffect(() => {
    const videoElement = videoRef.current;

    const handleMouseEnter = () => {
      setIsHovered(true);
      console.log('Mouse entered video player');
    };

    const handleMouseLeave = () => {
      setIsHovered(false);
      console.log('Mouse left video player');
    };

    if (videoElement) {
      videoElement.addEventListener('mouseenter', handleMouseEnter);
      videoElement.addEventListener('mouseleave', handleMouseLeave);
    }

    return () => {
      if (videoElement) {
        videoElement.removeEventListener('mouseenter', handleMouseEnter);
        videoElement.removeEventListener('mouseleave', handleMouseLeave);
      }
    };
  }, []);
```

```
return (
  <div
    ref={videoRef}
    style={{
      ...styles.videoPlayerContainer,
      backgroundColor: isDarkMode ? '#333' : '#f9f9f9',
    }}
  >
    <h2>Watch here to know about HKUST (GZ)!</h2>
    <iframe
      width="100%"
      height="315"
      src="https://player.bilibili.com/player.html?bvid=BV1iMzZYcEye&
      autoplay=0"
      title="Newest Video"
      allow="accelerometer; autoplay; clipboard-write; encrypted-media;
      gyroscope; picture-in-picture"
      allowFullScreen
      style={styles.videoIframe}
    ></iframe>
  </div>
);
```

Figure 9, 10. The refined VideoPlayer Component.

(isDarkMode refers to the assignment from the previous class and can be ignored for this context.)

Here, we use both the Effect Hook and the Ref Hook to design a behavior detector. The videoRef is initially set to null and is bound to the ref attribute of the in the return statement. After the component renders for the first time, the videoRef will be attached to the actual DOM element. Inside the useEffect, we define event handler functions for when the mouse enters or leaves the area, and we add event listeners to monitor the user's mouse behavior. If the user's mouse enters or leaves the area, the isHovered state is set to true or false, respectively, and a message is logged.

When you open the browser developer tools, if you move your mouse into or out of the videoPlayer, you will see logs shown in Figure 11.

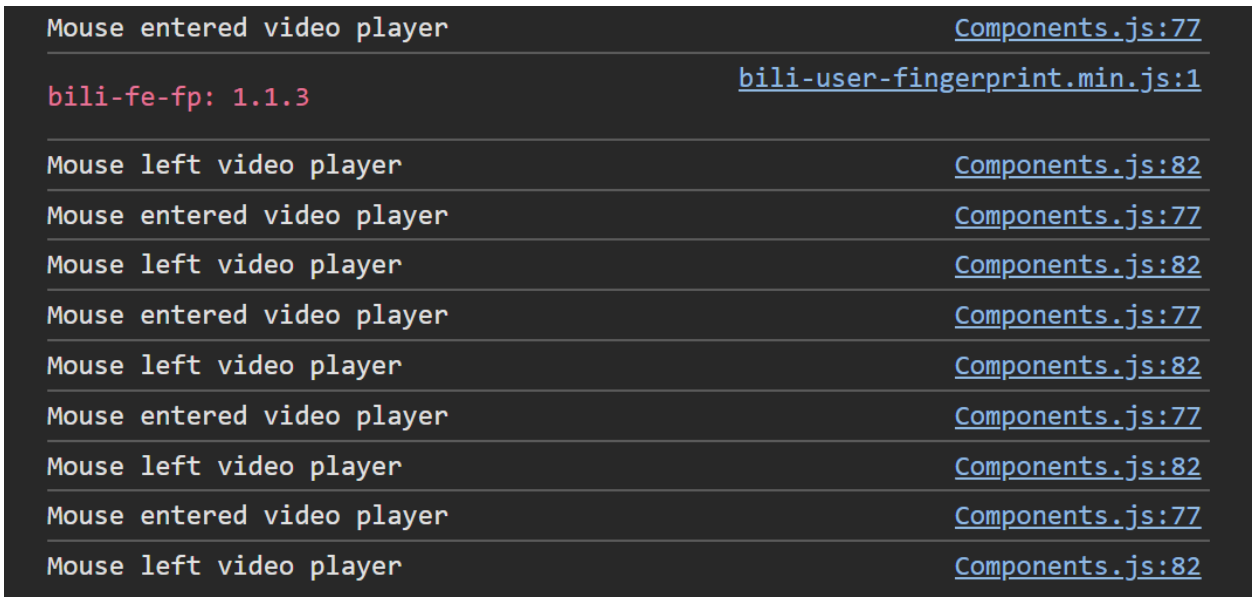


Figure 11. The behavior logs.

1.2.4. Context Hook

In our previous assignment, we designed a dark mode for the page and passed parameters to the styles of various components via props. Using the Context Hook, we can share global data among these components, therefore simplifying the code and avoiding the problem of prop drilling. The syntax for the Context Hook is shown below. React.createContext creates a new context object, AProvider represents the context provider, which supplies the value for the context, and the components within {children} can access the context value using useContext.

```
// 1. Initialize the context
const MyContext = React.createContext(defaultValue);

// 2. Provide the Context value (Provider)
```

```
const AProvider = ({ children }) => {
  const [state, setState] = useState(initialValue);

  return (
    <MyContext.Provider value={{ state, setState }}>
      {children}
    </MyContext.Provider>
  );
};

// 3. Use the Context (Consumer)
const MyComponent = () => {
  const contextValue = useContext(MyContext);
  // You can now use the values from contextValue
  return <div>{contextValue.state}</div>;
};

// 4. Use the Context Provider in the App
const App = () => {
  return (
    <AProvider> {/* Wrap the entire application in the Provider */}
      <MyComponent />
    </AProvider>
  );
};
```

Next, we will try to add a Context Hook to our website. We want to allow users to change the global font size of the website via buttons, thereby improving the website's accessibility. We will add this control functionality in the LeftProfile. First, we initialize the context:

```
const FontSizeContext = React.createContext({
  fontSize: 1,
  increaseFontSize: () => {},
  decreaseFontSize: () => {},
});
```

Figure 12. The FontSizeContext Initialization.

Next, we add the context provider:

```
const FontSizeContext = React.createContext({
  fontSize: 1,
  increaseFontSize: () => {},
  decreaseFontSize: () => {},
});

// Add Provider
const FontSizeProvider = ({ children }) => {
  const [fontSize, setFontSize] = useState(1);

  const increaseFontSize = () => setFontSize(prev => Math.min(prev + 0.1, 1.5));
  const decreaseFontSize = () => setFontSize(prev => Math.max(prev - 0.1, 0.8));

  return (
    <FontSizeContext.Provider value={{ fontSize, increaseFontSize, decreaseFontSize }}>
      {children}
    </FontSizeContext.Provider>
  );
};
```

Figure 12. The FontSize Context Provider.

Each time the "increase" or "decrease" button is clicked, the font size increases by 0.1 times, with a maximum of 1.5 times and a minimum of 0.8 times the original size. In the LeftProfile, we add buttons to adjust the font size, so that when they are clicked, the font size changes accordingly. At the same time, LeftProfile also acts as a consumer of the context (since the font size change is for all components), so we need to use useContext at the top level to retrieve the fontSize, increaseFontSize, and decreaseFontSize functions. Additionally, in the font styling section, we need to include fontSize: ${fontSize}rem to apply the modified font size to the text.

```
const LeftProfile = () => {
  const [stateCount, setStateCount] = useState(0);
  const refCount = useRef(0);
  const { fontSize, increaseFontSize, decreaseFontSize } = useContext(FontSizeContext);
```

```
  return (
    <div style={styles.leftProfile}>
      <img src="hkust(gz).jpg" alt="School Badge" style={styles.profileImage} />
      <div style={{ fontSize: `${fontSize}rem` }}>
      <h2>HKUST(GZ)</h2>
      <p>Nansha, Guangzhou, Guangdong, China.</p>
      <p>上一层、落两层，请使用楼梯</p>
      <div style={styles.socialLinks}>
        <a href="https://www.hkust-gz.edu.cn/" target="_blank" rel="noreferrer">
          Official Web
        </a>
        <a href="https://space.bilibili.com/16783101" target="_blank" rel="noreferrer">
          Bilibili
        </a>
        <a href="https://www.xiaohongshu.com/user/profile/640574e60000000029014ff1" target="_blank" rel="noreferrer">
          Xiaohongshu
        </a>
      </div>
      <div>
        <p>State Count: {stateCount}</p>
        <p>Ref Count: {refCount.current}</p>
        <button onClick={handleStateClick}>Add State</button>
        <button onClick={handleRefClick}>Add Ref</button>
      </div>
      <div style={styles.fontSizeControls}>
      <button onClick={decreaseFontSize} style={styles.fontSizeButton} aria-label="Decrease font size">A-</button>
      <button onClick={increaseFontSize} style={styles.fontSizeButton} aria-label="Increase font size">A+</button>
      </div>
      </div>
    </div>
  );
};
```

Figure 13, 14. The Refined LeftProfile.

In the MainPage, we also need to include the FontsizeProvider component.

```
const MainPage = () => {


  return (
    <div style={{ ...styles.pageContainer, ...themeStyles }}>
      {/* Left component */}
      <FontSizeProvider>
        <LeftProfile/>
      {/* right component */}
      <div style={styles.rightContent}>
        <AboutMe isDarkMode={isDarkMode} toggleTheme={() => setIsDarkMode(prev => !prev)} />
        <VideoPlayer isDarkMode={isDarkMode} />
        <Hubs hub={Hub} />
        <LatestNews time={current_time} news={received_news} />
        <Gallery/>
      </div>
      </FontSizeProvider>
    </div>
  );
};
```

Figure 15. The MainPage adds FontSizeProvider.

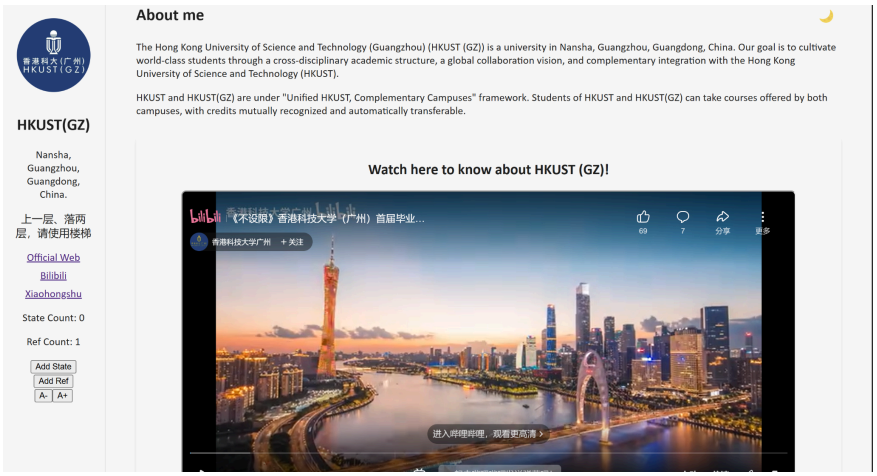
We can apply the global font size context to different components, which are similar to the setting in LeftProfile. This allows for consistent and manageable font size adjustments across the entire website.

```
const AboutMe = ({ isDarkMode, toggleTheme }) => {
  const { fontSize, increaseFontSize, decreaseFontSize } = useContext(FontSizeContext);
```

```
<div style={{ fontSize: `${fontSize}rem` }}>
<p>
  The Hong Kong University of Science and Technology (Guangzhou) (HKUST (GZ)) is a university in Nansha, Guangzhou, Guangdong,
  China. Our goal is to cultivate world-class students through a cross-disciplinary academic structure, a global collaboration
  vision, and complementary integration with the Hong Kong University of Science and Technology (HKUST).
</p>
<p>
  HKUST and HKUST(GZ) are under "Unified HKUST, Complementary Campuses" framework. Students of HKUST and HKUST(GZ) can take
  courses offered by both campuses, with credits mutually recognized and automatically transferable.
</p>
</div>
```

Figure 16, 17. The AboutMe applies the FontSizeContext.

Finally, we can adjust the font size using the A+ and A- buttons in the LeftProfile. These buttons allow users to increase or decrease the font size, providing a convenient way to customize the text for better readability and accessibility across the website.



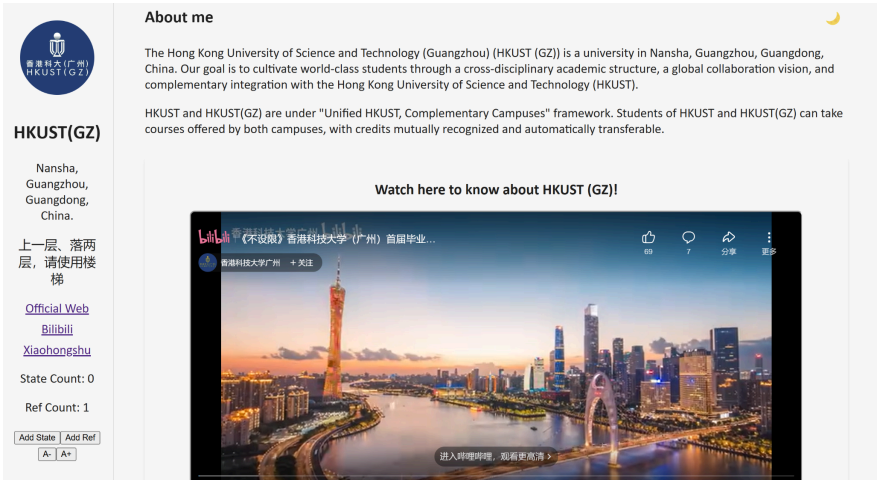


Figure 18, 19. The change of font size in the browser.

1.3. Custom hook

In addition to using React's built-in hooks, we can also create custom hooks. **In fact, custom hooks are combinations, encapsulations, or extensions of React's built-in hooks.** They are JS functions that **start with the word "use"**. Custom hooks allow you to extract and reuse stateful logic across different components. Here's an example with useCounter:

```
const useCounter = (initialValue = 0) => {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount(c => c + 1);
  const decrement = () => setCount(c => c - 1);

  return { count, increment, decrement };
};

const Profile(){
  const { count, increment } = useCounter(0);
  return (
    <div>
    <p>count:{count}</p>
    <div>
  )
}
```

This custom hook uses the built-in useState hook, defines the increment and decrement functions, and finally returns the three elements: count, increment, and decrement. In the Profile, we can use the useCounter to manage the counter logic. This makes the code more modular and reusable across different components.

Previously, we applied useEffect and useRef to implement behavior detection for the videoPlayer. We can now extract and encapsulate this part of the code into a custom hook called useHover. This hook accepts logName as a parameter and returns the ref object and signal. It also incorporates the mouseEnter, and mouseLeave. By creating the custom hooks, this detection functionality can be incorporated into any component, significantly enhancing flexibility and reusability.

```
const useHover = (logName = '') => {
  const elementRef = useRef(null);
  const [isHovered, setIsHovered] = useState(false);

  useEffect(() => {
    const element = elementRef.current;

    const handleMouseEnter = () => {
      setIsHovered(true);
      if (logName) console.log(`Mouse entered ${logName}`);
    };

    const handleMouseLeave = () => {
      setIsHovered(false);
      if (logName) console.log(`Mouse left ${logName}`);
    };

    if (element) {
      element.addEventListener('mouseenter', handleMouseEnter);
      element.addEventListener('mouseleave', handleMouseLeave);
    }

    return () => {
      if (element) {
        element.removeEventListener('mouseenter', handleMouseEnter);
        element.removeEventListener('mouseleave', handleMouseLeave);
      }
    };
  }, [logName]);

  return [elementRef, isHovered];
};
```

Figure 20. The custom hook useHover.

```jsx
const VideoPlayer = ({ isDarkMode }) => {
  const [videoRef, isHovered] = useHover('video player');

  return (
    <div
      ref={videoRef}
      style={{
        ...styles.videoPlayerContainer,
        backgroundColor: isDarkMode ? '#333' : '#f9f9f9',
      }}
    >
      <h2>Watch here to know about HKUST (GZ)!</h2>
      <iframe
        width="100%"
        height="315"
        src="https://player.bilibili.com/player.html?bvid=BV1iMzZYcEye&autoplay=0"
        title="Newest Video"
        allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
        allowFullScreen
        style={styles.videoIframe}
      ></iframe>
    </div>
  );
};
```

Figure 21. The VideoPlayer applies the userHover.

```
Mouse entered video player                    Components.js:107
Mouse left video player                       Components.js:112
Mouse entered hub                             Components.js:107
Mouse left hub                                Components.js:112
Mouse entered latest news                     Components.js:107
Mouse left latest news                        Components.js:112
```

Figure 22. The results of useHover.

Task2. Router

2.1 What is Router in React?

React Router is the standard library for implementing routing (page navigation) in React applications. It allows us to create multiple "pages" within a single-page application (SPA). For example, on personal websites, you might see options such as "about" and "projects." When you click on these options, the content of the page updates accordingly without reloading the entire page. This is the role of React Router.

2.2 Key Component in React Router

We first implement the following code in App.js. Here, AComponent is a component in Lab 3, when studying bind and unbind, and it is stored in test.js. We will then explore the roles of each router component.

```jsx
import logo from './logo.svg';
import './App.css';
import MainPage from './Components';
import AComponent from './test';
import { BrowserRouter, Routes, Route, Link, NavLink } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>{' '}
        <Link to="/test">Test</Link>
      </nav>
      <Routes>
        <Route path="/" element={<MainPage />} />
        <Route path="/test" element={<AComponent />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Figure 23. The Router used in App.js.

BrowserRouter: This is typically used as the top-level component to wrap the entire application, enabling routing functionality for all child components. BrowserRouter enables users to navigate through the app, while the URL reflects the current page location. Additionally, it provides a routing-related context to all child components. This context includes routing information (e.g., current path, matching rules), allowing nested components to easily access this information.

Route: It defines the mapping between a URL path and the component to be rendered. Its syntax is as follows:

```
<Route path="/test" element={<AComponent />} />
```

It means that when the URL "website/test" entered, the system will render the AComponent.

Routes: Contains all the Route components and only renders the first Route that matches the current URL. Without the Routes component, all matching Routes would be rendered. For example, if you enter "website/test", both "/" and "/test" might match, causing two components to render simultaneously.

Link: The basic navigation component used to create links. After implementing it, you will see "Home" and "Test" buttons at the top of the page, and clicking them will navigate to different URLs.

We can also use NavLink, which is a special version of Link. It can apply styles when the link is active. For example, we can add special CSS to indicate which page option is currently displayed. Due to space constraints, we won't go into detailed examples here, but this feature is useful for visually highlighting the active navigation item.

2.3 Hooks in React Router

In React Router, there are also some commonly used hooks, such as useLocation and useNavigate. useLocation can check the current page path, while useNavigate can be used to perform page navigation and pass state data.

Next, we utilize these components and hooks to separate the LatestNews into an independent News page (to achieve this project, you can refer to the new CSS settings provided in the appendix). The original page will serve as the Home page. We first create a NavigationBar to construct the navigation bar at the top, and we will also move the Dark Mode/Light Mode toggle button here (removing the buttons in AboutMe).

```
const NavigationBar = ({ isDarkMode, toggleTheme, currentPath, navigate }) => {
  return (
    <div style={styles.navigationBar}>
      <button style={styles.navButton} onClick={() => navigate('/')}>Home</button>
      <button style={styles.navButton} onClick={() => navigate('/news')}>News</button>
      <button onClick={toggleTheme} style={styles.themeToggleButton}>{isDarkMode ? '🌞' : '🌙'}
      </button>
    </div>
  );
};
```

Figure 24. The NavigationBar Component.

Next, we create a HomeContent component to display all other components except for the LatestNews.

```
const HomeContent = ({ isDarkMode}) => {
  const Hub = [
    { imgSrc: 'FUNCTION.jpg', title: 'Function Hub' },
    { imgSrc: 'INFO.jpg', title: 'Information Hub' },
    { imgSrc: 'SOCIETY.jpg', title: 'Society Hub' },
    { imgSrc: 'SYSTEM.jpg', title: 'System Hub' }
  ];

  return (
    <div>
      <AboutMe isDarkMode={isDarkMode}/>
      <VideoPlayer isDarkMode={isDarkMode} />
      <Hubs hub={Hub} />
      <Gallery/>
    </div>
  );
};
```

Figure 25. The HomeContent Components.

In the MainPage, we use useLocation to obtain the current webpage's URL and implement navigation using useNavigate. If `location.pathname` is `"/news"`, then isNewsPage will be set to true.

```
const MainPage = () => {
  const [isDarkMode, setIsDarkMode] = useState(false);
  const location = useLocation();
  const navigate = useNavigate();
  const isNewsPage = location.pathname === '/news';
```

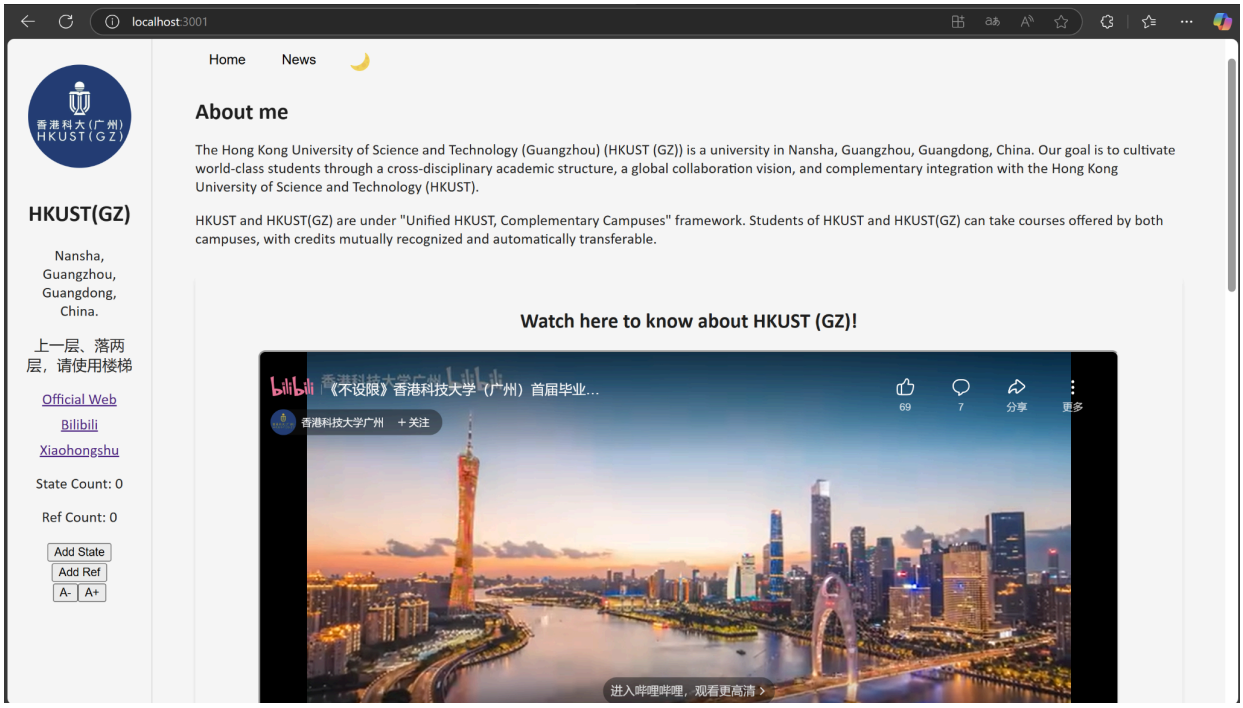
Figure 24. The MainPage applies useLocation and useNavigate.

The return statement of `MainPage` should be modified as follows: when `isNewsPage` is `true`, render the `LatestNews` component; otherwise, render the `HomeContent` component.

```jsx
return (
  <div style={{ ...styles.pageContainer, ...themeStyles }}>
    <FontSizeProvider>
      <LeftProfile/>
      <div style={styles.rightContent}>
        <NavigationBar isDarkMode={isDarkMode} toggleTheme={() => setIsDarkMode(prev => !prev)} currentPath={location.pathname} navigate=
        {navigate} />
        {isNewsPage ? (
          <LatestNews time={current_time} news={received_news} />
        ) : (
          <HomeContent isDarkMode={isDarkMode} />
        )}
      </div>
    </FontSizeProvider>
  </div>
);
```

Figure 25. The Refined MainPage.

Finally, we can view both the Home and News pages in the browser, and clicking on them we will see different pages.



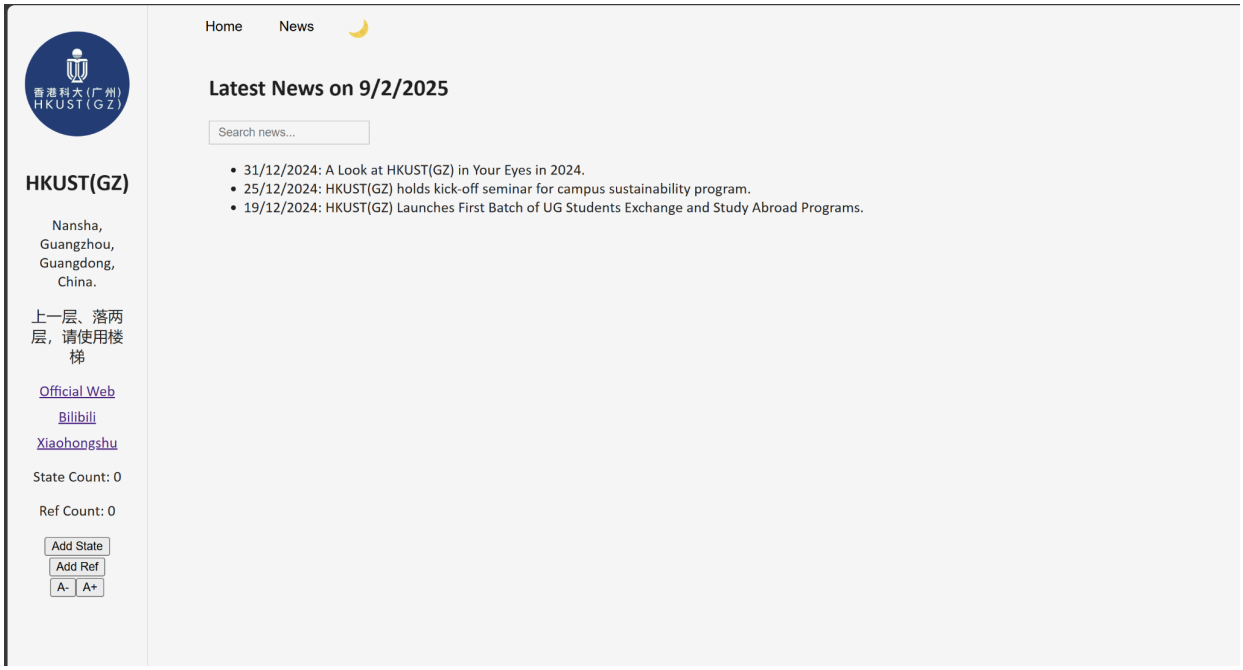


Figure 26, 27. The Home and News in Websites.